

Project Brief: "Project Nexus" – Enterprise Internal Developer Platform (IDP)

Role: Senior DevOps / Platform Engineer
Objective: Architect and implement a production-grade, highly scalable Internal Developer Platform (IDP) on AWS Kubernetes to support a 30-microservice ecosystem. The goal is to eliminate configuration drift, automate scaling, and provide a zero-friction deployment experience for software engineers.

1. System Context & Workloads

The platform must support and route traffic for 30 distinct microservices operating across three different language ecosystems:

- **10x Go Services:** High-throughput API gateways and real-time transaction routers.
- **10x Python Services:** AI/ML inference workloads and async background workers (Celery).
- **10x TypeScript (Node.js) Services:** User-facing backend APIs and third-party integrations.

2. Approved Technology Stack

- **Cloud Provider:** AWS
- **Infrastructure as Code (IaC):** Terraform or OpenTofu
- **Container Orchestration:** Amazon EKS (Kubernetes)
- **Compute Provisioning:** Karpenter (replacing standard Cluster Autoscaler)
- **Networking & CNI:** Cilium (eBPF-based)
- **CI/CD Pipeline:** GitHub Actions + ArgoCD (GitOps)
- **Observability:** OpenTelemetry (OTel) + LGTM Stack (Loki, Grafana, Tempo, Mimir) or Datadog
- **Security & Governance:** Kyverno (Policy), Falco (Runtime Security), Trivy (Scanning)
- **Developer Portal:** Backstage (Spotify)

3. Scope of Work & Deliverables

Phase 1: Reproducible Cloud Foundation (IaC)

- **Requirement:** Write modular Terraform/OpenTofu code to provision the underlying AWS infrastructure (VPC, private/public subnets, NAT Gateways, EKS Cluster, IAM OIDC providers).
- **State Management:** State must be stored securely in S3 with DynamoDB state locking.
- **Deliverable:** A dedicated infrastructure repository where a single automated pipeline can tear down and rebuild the entire staging environment in under 30 minutes.

Phase 2: GitOps & Cluster Bootstrapping

- **Requirement:** Implement an "App of Apps" GitOps pattern using ArgoCD.
- **Execution:** Manual `kubectl` commands in production are strictly prohibited. The cluster must pull its desired state from a Git repository.
- **Deliverable:** Automated installation of all cluster add-ons (External-DNS, Cert-Manager, Karpenter, Cilium) immediately upon cluster creation via ArgoCD.

Phase 3: Compute & Networking (eBPF)

- **Requirement:** Configure Karpenter to provision Node/Spot instances dynamically based on pod requirements, optimizing for AWS cost.
- **Requirement:** Deploy Cilium as the CNI. Implement strict L3/L4/L7 Network Policies (Default Deny). Example: Node.js frontend pods may only communicate with Go API gateways; Python ML workers cannot access the public internet.

Phase 4: Unified Observability Mesh

- **Requirement:** Standardize logging, metrics, and tracing across all 30 microservices using OpenTelemetry.
- **Execution:**
 - Set up OTel Collectors in the cluster.
 - Implement Distributed Tracing so a single request can be tracked across the Node.js API, Go router, and Python worker in a single trace view.
- **Deliverable:** Pre-configured Grafana dashboards for cluster health (CPU/Memory/Network) and APM (Application Performance Monitoring) for the microservices.

Phase 5: DevSecOps & Shift-Left Security

- **Requirement:** Integrate security scanning and runtime protection.
- **Execution:**
 - Container image scanning via Trivy in the CI pipeline (blocking critical CVEs).
 - Enforce Kubernetes admission policies via Kyverno (e.g., block containers running as root, enforce resource quotas).
 - Deploy Falco for real-time threat detection (e.g., alert if a shell is spawned inside a pod).
- **Deliverable:** Security alerts routed to a dedicated Slack/Teams channel.

Phase 6: Developer Experience (Platform Engineering)

- **Requirement:** Deploy and configure Spotify's Backstage.
 - **Execution:** Create software templates for the 3 supported languages.
 - **Deliverable:** A developer must be able to click "Create New Go Service" in Backstage, which automatically creates the GitHub repo, provisions the CI/CD pipeline, and deploys a "Hello World" container to the dev cluster without DevOps intervention.
-

4. Definition of Done (DoD)

1. All infrastructure is defined in code and version-controlled.
2. Any of the 30 microservices can be deployed or rolled back purely by merging a Pull Request.
3. A failed deployment does not cause downtime (rolling updates or blue/green configured).
4. Logs, metrics, and traces for all three languages are accessible from a single pane of glass.
5. System architecture and incident runbooks are documented in the team's wiki.